Machine Learning

Pain Level Classification

Name: - Hasnaa Nageh Ahmed

ID: - 12300560

Introduction

This project aims to classify human pain levels using machine learning algorithms. Pain is a complex physiological and psychological experience that is difficult to measure objectively. Traditional pain assessment relies on self-reporting, which can be subjective and inaccurate. Therefore, using biosensor data to automatically detect and classify pain levels provides a more objective and reliable approach.

In this project, we used a dataset collected from wearable biosensors that measure various physiological signals from the human body. The dataset contains 96,000 records from multiple patients, with 7 input features and one target variable representing the pain scale.

Six machine learning algorithms were implemented and compared in this project:

- Decision Tree
- Random Forest
- Linear Regression
- Logistic Regression
- K-Nearest Neighbors (KNN)
- Naïve Bayes

Each algorithm was implemented in two ways — using the built-in scikit-learn library and from scratch using Python and NumPy. The results were then compared to evaluate the performance of each algorithm.

The main objectives of this project are:

- To implement and compare six machine learning algorithms
- To evaluate each algorithm using accuracy and classification metrics
- To identify the best performing algorithm for pain level classification
- To compare built-in library results with from-scratch implementations

Dataset Description

The dataset contains 96,000 records and 9 columns. It was collected at a frequency of 4Hz, meaning 4 readings per second were recorded from each patient.

Features Description:

The dataset contains the following features:

- **person_ID:** A unique identifier for each patient (e.g., P001, P002). This column was removed before training as it is not a numerical feature.
- **acc_x, acc_y, acc_z:** Accelerometer readings measuring body movement in three directions (x, y, z). These signals capture how the patient moves in response to pain.
- **EDA (Electrodermal Activity):** Measures the electrical conductance of the skin. When a person experiences pain or stress, the skin produces more sweat, which changes its conductance.
- **BVP (Blood Volume Pulse):** Measures the blood flow through the blood vessels. Pain affects the cardiovascular system and changes the blood flow pattern.
- **HR (Heart Rate):** Measures the number of heartbeats per minute. Pain typically causes an increase in heart rate.
- **temp:** Measures the skin temperature. Pain can cause changes in skin temperature due to changes in blood flow.
- **pain_scale:** This is the target variable, representing the patient's self-reported pain level. It ranges from 1 to 8.

Dataset Statistics:

Property	Value
Total Records	96,000
Total Features	7
Target Classes	8 (1 to 8)
Missing Values	None
Data Type	Float64 / Int64

Pain Scale Distribution:

Pain Level	Count
1	9,600
2	9,120

3	13,920
4	17,280
5	10,560
6	16,320
7	7,680
8	11,520

The dataset is slightly imbalanced, with pain levels 4 and 6 having the highest number of records, and pain level 7 having the lowest. However, the difference is not significant enough to require special handling.

Algorithms Description

1. Decision Tree

Decision Tree is a supervised machine learning algorithm that works by splitting the data into branches based on feature values. It asks a series of questions about the data, and each answer leads to another question or a final decision.

The algorithm selects the best feature to split the data at each node by calculating the **Entropy** and **Information Gain**. The split that results in the highest information gain is chosen.

Parameters used in this project:

- criterion = 'entropy'
- max_depth = None
- min_samples_split = 2

2. Random Forest

Random Forest is an ensemble learning algorithm that builds multiple decision trees and combines their results. Each tree is trained on a random sample of the data, and the final prediction is made by majority voting.

This approach reduces overfitting and improves accuracy compared to a single decision tree.

Parameters used in this project:

- n_estimators = 100
- criterion = 'entropy'
- random_state = 42

3. Linear Regression

Linear Regression is a supervised machine learning algorithm that predicts a continuous value by fitting a straight line through the data points. The equation is:

$$y = w_1x_1 + w_2x_2 + ... + w_nx_n + b$$

Although Linear Regression is not designed for classification, it was used in this project by rounding and clipping the predicted values to the nearest pain scale value between 1 and 8.

The algorithm minimizes the **Mean Squared Error (MSE)** using the Normal Equation:

$$W = (X^TX)^{-1} X^Ty$$

4. Logistic Regression

Logistic Regression is a classification algorithm that predicts the probability of each class using the **Softmax** function. Despite its name, it is used for classification, not regression.

The algorithm calculates a weighted sum of the input features and passes it through the Softmax function to produce probabilities for each class:

$$P(class) = softmax(w_1x_1 + w_2x_2 + ... + b)$$

The class with the highest probability is selected as the final prediction.

Parameters used in this project:

- max_iter = 1000
- random_state = 42

5. K-Nearest Neighbors (KNN)

KNN is a simple and intuitive algorithm that classifies a new data point based on the majority class of its K nearest neighbors in the training data.

The distance between data points is calculated using the **Euclidean Distance:**

$$d = \sqrt{(x_1-y_1)^2 + (x_2-y_2)^2 + \dots}$$

KNN does not have a training phase — it stores all training data and performs calculations only when a new prediction is needed. This makes it a **Lazy Learner.**

Parameters used in this project:

- n_neighbors = 5

6. Naïve Bayes

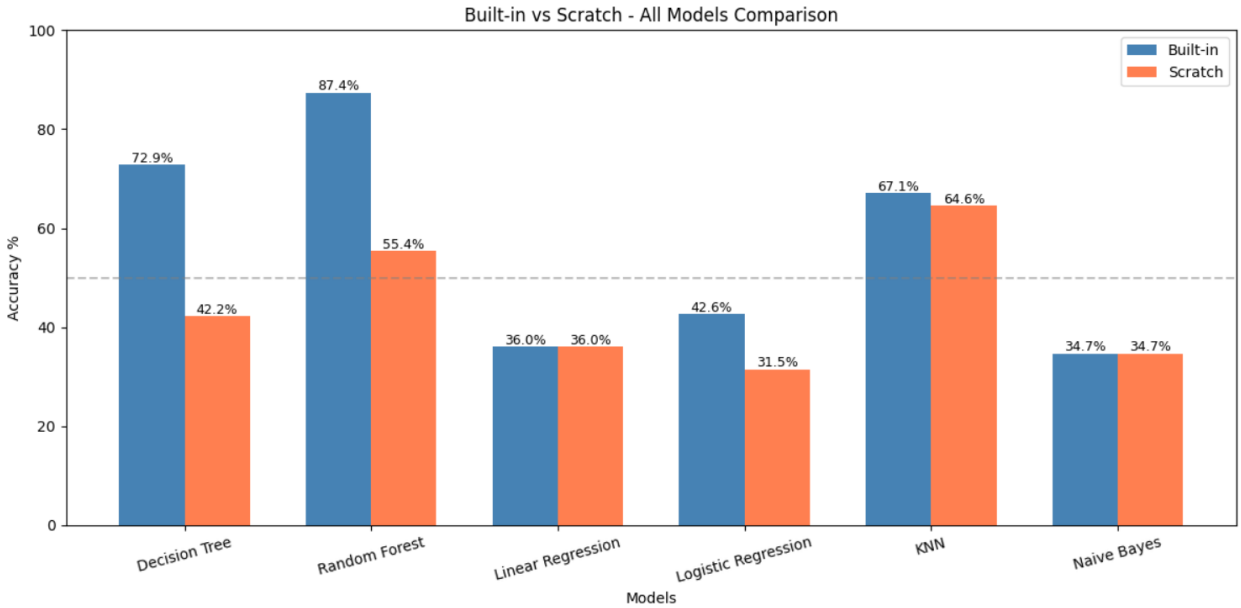
Naïve Bayes is a probabilistic classification algorithm based on **Bayes' Theorem.** It calculates the probability of each class given the input features and selects the class with the highest probability.

$$P(\text{Class} \mid \text{Features}) \propto P(\text{Class}) \times P(\text{Features} \mid \text{Class})$$

It is called "Naïve" because it assumes that all features are independent of each other, which is rarely true in practice but works well in many cases.

In this project, **Gaussian Naïve Bayes** was used, which assumes that the features follow a normal distribution.

Results and Comparison



Built-in vs Scratch

Algorithm	Built-in	Scratch
Decision Tree	72.86%	42.20%
Random Forest	87.41%	55.35%
Linear Regression	36.05%	36.05%
Logistic Regression	42.61%	31.51%
KNN	67.07%	64.60%
Naive Bayes	34.68%	34.68%

Code & Dataset

https://github.com/Hasnaa-Nageh/Machine-Learning-Project.git